// АЛГОРИТМИЗАЦИЯ И ПРОГРАММИРОВАНИЕ · ЛЕКЦИЯ 7

Составные типы данных: массивы

Память, слайсы и списки, linked list, ассоциативные массивы, ссылочные типы, list и dict в Python



СПбГУПТД · кафедра цифровых и аддитивных технологий

// ПЛАН

О чём сегодня

01 Массив и устройство памяти

02 Динамические массивы: slice и list

03 Связный список против массива

04 Значения и ссылки

05 Ассоциативные массивы: dict и map

06 Операции над list в Python

07 Перебор: for, enumerate, items

01

Массив и устройство памяти

Почему доступ по индексу — это одна формула

// 01 · МАССИВ

Что такое массив

Массив — набор элементов одного типа, которые лежат в памяти подряд, без промежутков. К элементу обращаются по номеру — индексу.

Один размер

Все элементы занимают одинаковое число байт: int32 — 4, float64 — 8.

Непрерывность

Один сплошной блок памяти. Следующий элемент — сразу за предыдущим.

Индекс с нуля

Индекс — это смещение от начала: первый элемент сдвинут на 0.

// 01 · ПАМЯТЬ

Как массив лежит в памяти




```
адрес(a[i]) = base + i × size
```

a[3] → 0x1000 + 3 × 4 = 0x100C

O(1)

Неважно, 10 элементов в массиве или 10 миллионов — одно умножение и одно сложение.

// 01 · GO

Массив в Go: фиксированная длина

```go
var a [5]int  // [0 0 0 0 0]
b := [3]string{"x", "y", "z"}
a[2] = 10
fmt.Println(len(a))  // 5

c := a  // копия всех 5 элементов
c[0] = 99  // a не изменится

// a[7] = 1 → ошибка компиляции
```

Длина — часть типа

[3]int и [5]int — разные типы, размер известен при компиляции.

Массив — значение

Присваивание и передача в функцию копируют все элементы.

А в Python?

Встроенного «жёсткого» массива нет: есть list, а для чисел — array и numpy.

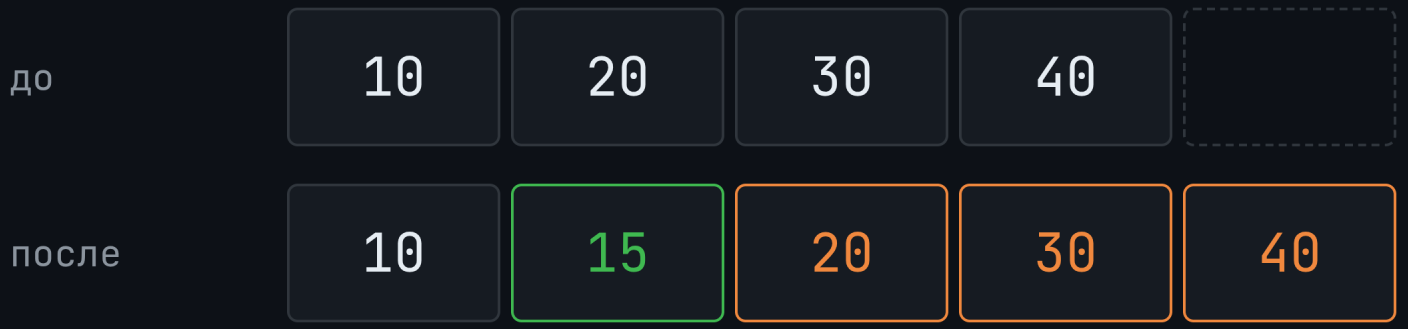
// 01 · ОПЕРАЦИИ

Базовые операции и их стоимость

Операция	Сложность	Почему
Чтение / запись a[i]	O(1)	одна формула адреса
Поиск значения	O(n)	перебор подряд; O(log n), если отсортирован
Вставка в середину	O(n)	хвост сдвигается вправо
Удаление из середины	O(n)	хвост сдвигается влево
Добавление в конец	O(1)*	* если есть свободное место
Обход всех элементов	O(n)	каждый элемент по одному разу

// 01 · ВСТАВКА

Вставка в середину — это сдвиг



Что произошло

Чтобы вставить 15 на позицию 1, элементы 20, 30, 40 переехали на ячейку вправо.

Худший случай

Вставка в начало сдвигает все n элементов — отсюда O(n). Удаление работает так же, только влево.

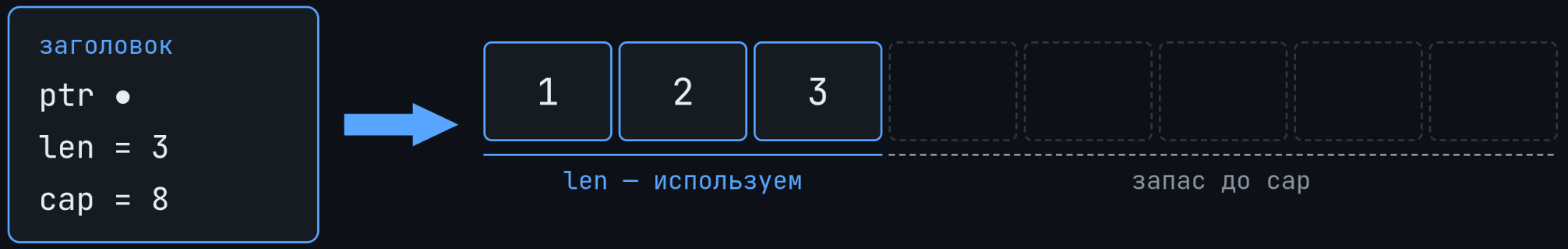
02

Динамические массивы

Как получить массив, который умеет расти: slice в Go и list в Python

// 02 · SLICE

Слайс в Go: окно в массив




```
s := make([]int, 3, 8)
// len 3, cap 8
```

Слайс — маленькая структура из трёх полей. Сами данные лежат в обычном массиве, а слайс показывает на его кусок.

// 02 · APPEND

append и рост ёмкости

```
s := []int{}
for i := 0; i < 10; i++ {
    s = append(s, i)
    fmt.Println(len(s), cap(s))
}
// 1 1 · 2 2 · 3 4 · 5 8 · 9 16
```

Место есть

Пишем в следующую ячейку — O(1).

Места нет

Выделяем новый массив побольше, копируем всё туда — O(n).

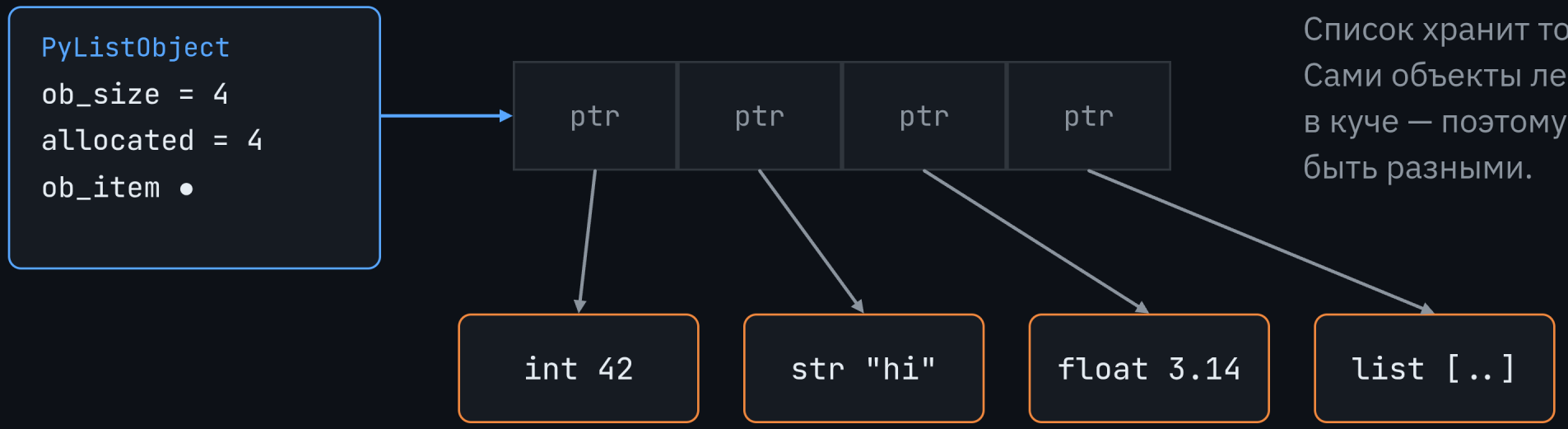
В среднем

Копирования редкие, поэтому append — амортизированное O(1).

cap: 1 → 2 → 4 → 8 → 16 → …

// 02 · PYTHON LIST

list в Python: массив указателей



Список хранит только адреса. Сами объекты лежат где угодно в куче — поэтому типы могут быть разными.

Цена гибкости: каждый элемент — отдельный объект. Небольшой int — это 28 байт плюс 8 байт указателя. Для больших числовых данных берут array или numpy.

// 02 · PYTHON LIST

Python тоже резервирует место

```
import sys
a = []
for i in range(10):
    a.append(i)
    print(len(a), sys.getsizeof(a))

# 1..4 → 88 байт
# 5..8 → 120 байт
# 9..  → 184 байта
```

Ступеньки

Размер растёт скачками: память выделяется с запасом, а не на каждый append.

Мягкий рост

Новая ёмкость ≈ n + n/8 + 6: примерно +12,5 %, а не ×2, как в Go.

Итог

append — O(1) амортизированно, insert(0, x) — O(n).

03

Связный список

Другая идея: элементы не обязаны лежать рядом

// 03 · LINKED LIST

Связный список (linked list)




```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

head = Node(10, Node(20, Node(30)))
```

Узел = значение + ссылка на следующий узел.

Узлы разбросаны по памяти — формулы адреса нет, к i-му идём от головы.

Вставка после известного узла — перекинуть две ссылки, O(1).

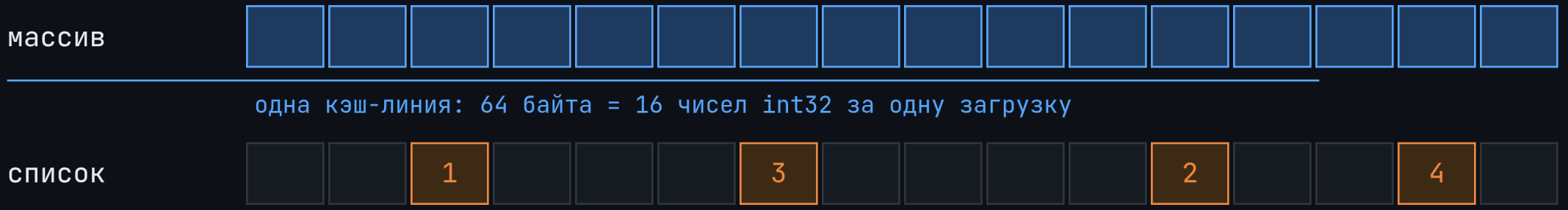
// 03 · СРАВНЕНИЕ

Массив против связного списка

Операция	Динамический массив	Связный список
Доступ по индексу	O(1)	O(n) — идём от головы
Вставка в начало	O(n) — сдвиг	O(1)
Вставка в середину	O(n) — сдвиг	O(1), но поиск места O(n)
Добавление в конец	O(1) аморт.	O(1), если хранить tail
Память на элемент	только данные	данные + ссылки
Кэш процессора	работает отлично	почти не помогает

// 03 · КЭШ

Почему массив на практике быстрее



Порядки величин

Чтение из кэша — около 1 нс, из оперативной памяти — порядка 100 нс.

Вывод

list, slice, vector, ArrayList — динамические массивы. Список — для частых вставок в известном месте.

04

Значения и ссылки

Что на самом деле копирует оператор =

// 04 · ТИПЫ

Значимые и ссылочные типы

Значимый тип

Переменная хранит **сами данные**. Присваивание — полная копия.

`Go: int, float64, bool, массив [N]T, struct`

Ссылочный тип

Переменная хранит **адрес данных**. Присваивание копирует адрес — данные общие.

`Go: slice, map, указатели`

А в Python всё — ссылки

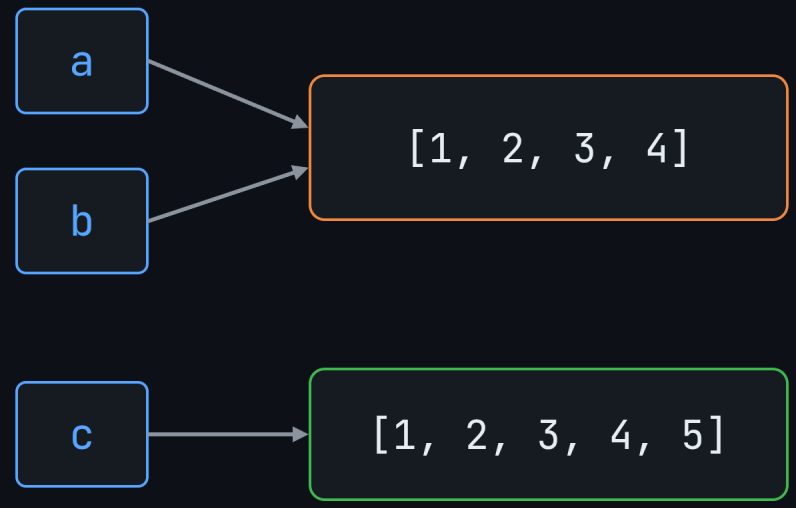
Любая переменная — имя, привязанное к объекту. Важно другое: объект изменяемый (list, dict, set) или неизменяемый (int, str, tuple). С неизменяемыми общая ссылка не опасна.

// 04 · ПСЕВДОНИМЫ

Две переменные — один список

```
a = [1, 2, 3]
b = a          # не копия!
b.append(4)
print(a)       # [1, 2, 3, 4]
print(a is b) # True

c = a.copy()  # или a[:], list(a)
c.append(5)
print(a)       # [1, 2, 3, 4]
```



Оператор = не создаёт новый объект, а даёт ему ещё одно имя. is проверяет, тот ли это объект; == — равны ли значения.

// 04 · КОПИРОВАНИЕ

Поверхностная и глубокая копия

```
grid = [[0] * 3] * 3
grid[0][0] = 1
print(grid)  # [[1,0,0],[1,0,0],[1,0,0]]
grid = [[0] * 3 for _ in range(3)]  # ок

import copy
m = [[1, 2], [3, 4]]
s = m.copy()          # поверхностная
d = copy.deepcopy(m) # глубокая
m[0][0] = 99
# s[0][0] == 99, d[0][0] == 1
```

*** 3 копирует ссылки**

Три строки матрицы — один и тот же внутренний список.

copy(), [:]

Новый внешний список, но вложенные объекты общие.

deepcopy

Рекурсивно копирует всё дерево объектов.

// 04 · GO

Go: массив копируется, слайс — нет

```
arr := [3]int{1, 2, 3}
arr2 := arr    // копия данных
arr2[0] = 99   // arr: [1 2 3]

sl := []int{1, 2, 3}
sl2 := sl      // копия заголовка
sl2[0] = 99    // sl: [99 2 3]

sub := sl[0:2]        // та же память
sub = append(sub, 7) // sl: [99 2 7]
```

Копия слайса

Копируются три поля: ptr, len, cap. Массив под ними общий.

Срез слайса

Делит память с исходным; append может затереть чужие данные.

Нужна копия

copy(dst, src) или slices.Clone(s).

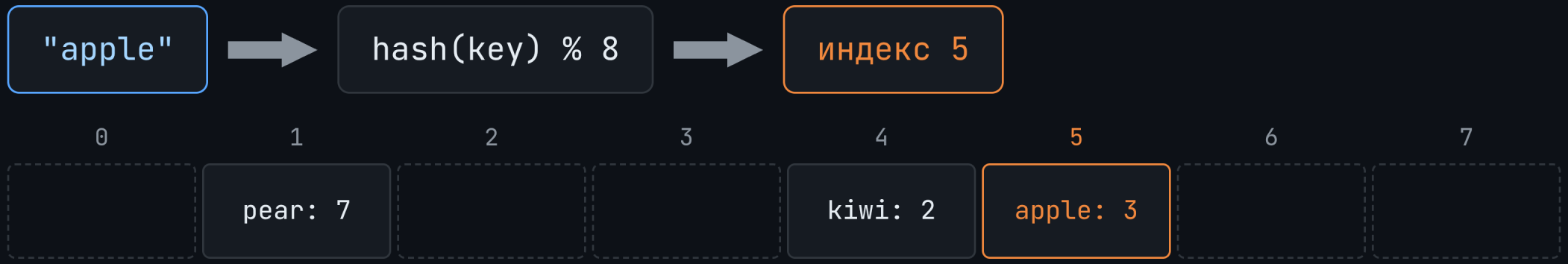
05

Ассоциативные массивы

Индекс — не номер, а любой ключ

// 05 · ХЕШИРОВАНИЕ

Ключ → значение через хеш



O(1) в среднем

Ключ превращается в индекс — дальше это обычный массив.

Коллизии

Два ключа в одну ячейку — цепочки или поиск соседней свободной.

Ключ неизменяем

str, int, tuple — можно; list — нельзя: его хеш поменялся бы.

// 05 · DICT И MAP

dict в Python и map в Go

Python

```python
ages = {"Аня": 19, "Борис": 20}
ages["Вика"] = 18  # добавить
ages["Аня"] += 1   # изменить
ages.get("Глеб", 0) # 0
ages["Глеб"]       # KeyError
del ages["Борис"]
"Аня" in ages      # True
```

Go

```go
ages := map[string]int{
    "Аня": 19, "Борис": 20,
}
ages["Вика"] = 18
ages["Аня"]++
v, ok := ages["Глеб"] // 0 false
delete(ages, "Борис")
```

С Python 3.7 dict сохраняет порядок вставки.

Порядок обхода map в Go намеренно случайный.

06

list в Python: операции

Индексы, срезы, методы и их стоимость

// 06 · ИНДЕКСЫ И СРЕЗЫ

Создание, индексы, срезы

```
a = [10, 20, 30, 40, 50]
a[0]    # 10
a[-1]   # 50 — с конца
a[1:4]  # [20, 30, 40]
a[:2]   # [10, 20]
a[::2]  # [10, 30, 50]
a[::-1] # [50, 40, 30, 20, 10]
len(a)  # 5
```



Подсвечен срез a[1:4]: start включается, stop — нет. Срез всегда создаёт новый список.

a[start : stop : step]

// 06 · МЕТОДЫ

Методы list и их сложность

Операция	Что делает	Сложность
a.append(x)	добавить в конец	O(1)*
a.extend(b)	добавить все элементы b	O(k)
a.insert(i, x)	вставить по индексу	O(n)
a.pop()	убрать и вернуть последний	O(1)
a.pop(i)	убрать по индексу	O(n)
a.remove(x)	убрать первое вхождение x	O(n)
x in a, a.index(x)	поиск значения	O(n)
a.sort(), sorted(a)	сортировка	O(n log n)
a.reverse()	развернуть на месте	O(n)

// 06 · ПОЛЕЗНОЕ

Операторы и встроенные функции

```
a = [3, 1, 2]
b = a + [4, 5]     # новый список
c = [0] * 4        # [0, 0, 0, 0]
sum(a), min(a), max(a) # 6 1 3
sorted(a)          # [1, 2, 3], a прежний
a.sort()           # меняет a, вернёт None
a == [1, 2, 3]     # True
del a[0]           # [2, 3]
```

sort() или sorted()

Метод меняет список на месте. Функция возвращает новый и работает с любой коллекцией.

Частая ошибка

`a = a.sort()`

теперь в a лежит None.

07

Перебор коллекций

for, enumerate, zip, keys, values, items

// 07 · FOR

Цикл for по списку

```python
fruits = ["яблоко", "груша", "слива"]

# по элементам
for f in fruits:
    print(f)

# по индексам — так делают редко
for i in range(len(fruits)):
    print(i, fruits[i])
```

for … in

Берёт элементы по очереди через итератор. Индекс не нужен.

Нужна позиция?

Вместо range(len(…)) — enumerate, на следующем слайде.

То же в Go

```go
for i, f := range fruits {
    fmt.Println(i, f)
}
```

// 07 · ENUMERATE И ZIP

enumerate и zip

```
for i, f in enumerate(fruits):
    print(i, f)     # 0 яблоко …

for i, f in enumerate(fruits, start=1):
    print(f"{i}. {f}") # 1. яблоко

prices = [80, 120, 95]
for f, p in zip(fruits, prices):
    print(f, p)     # яблоко 80 …
```

enumerate

Выдаёт пары (индекс, элемент). Пара сразу распаковывается в i, f.

start=

С какого числа считать — удобно для нумерованных списков.

zip

Идёт параллельно по нескольким коллекциям и останавливается на самой короткой.

// 07 · СЛОВАРИ

Перебор словаря: keys, values, items

```
stock = {"яблоко": 12, "груша": 0, "слива": 7}

for name in stock:          # = stock.keys()
    print(name)

for count in stock.values():
    print(count)

for name, count in stock.items():
    print(f"{name}: {count}")
```

`keys()`	ключи
`values()`	значения
`items()`	пары (ключ, значение)

Это представления (views): они видят изменения словаря. В Go то же даёт for k, v := range m.

// 07 · COMPREHENSIONS

Генераторы списков и словарей

```
nums = [1, 2, 3, 4, 5, 6]
squares = [x * x for x in nums]          # [1, 4, 9, 16, 25, 36]
evens = [x for x in nums if x % 2 == 0]  # [2, 4, 6]
in_stock = {k: v for k, v in stock.items() if v > 0}
# {'яблоко': 12, 'слива': 7}
```

Шаблон

`[выражение for x in коллекция if условие]`

Зачем

Короче цикла с append и обычно быстрее.

Без фанатизма

Не читается в одну строку — пишите обычный цикл.

// 07 · ЛОВУШКА

Не меняйте коллекцию во время обхода

```
nums = [1, 2, 2, 3]
for x in nums:
    if x == 2:
        nums.remove(x)
print(nums)  # [1, 2, 3] — двойка выжила

nums = [x for x in nums if x != 2]  # ок

for k in list(d):  # обходим копию ключей
    if d[k] == 0:
        del d[k]
```

Список

После удаления элементы сдвигаются, и цикл молча перепрыгивает через соседа.

Словарь

Python сразу бросает RuntimeError: размер изменился во время обхода.

Решение

Собрать новую коллекцию или обходить копию.

// ИТОГИ

Что нужно унести с лекции

01

Массив — непрерывная память, поэтому a[i] за O(1).

02

slice и list держат запас ёмкости: append — O(1) амортизированно.

03

Связный список: дешёвая вставка, дорогой доступ, плохой кэш.

04

= копирует ссылку: list, dict, slice после присваивания общие.

05

dict и map: хеш ключа → индекс, O(1) в среднем, ключ неизменяем.

06

Обход: for, enumerate, zip, items(). В цикле коллекцию не меняем.

// САМОПРОВЕРКА

Вопросы для самопроверки

1. Почему время доступа к a[i] не зависит от длины массива?
2. Чем len отличается от cap у слайса в Go?
3. Что выведет код: a = [1, 2]; b = a; b += [3]; print(a)?
4. Почему [[0] * 3] * 3 — плохой способ создать матрицу?
5. Почему list нельзя сделать ключом словаря, а tuple можно?
6. В каких задачах связный список выгоднее массива?